C 言語を用いたウェブサービスの実装

簡易的な IRC サーバおよびクライアントの作成

Jonghyeon PARK

2026-01

佐賀大学 理工学部 特別聴講学生

25931028@edu.cc.saga-u.ac.jp; jonghyeon@jnu.ac.kr

ウェブサーバの C 言語実装

課題要件

- クライアントとサーバの両方から通信を開始できること
- 通信プロトコルとして TCP/IP（または UDP/IP）を使用すること
- メッセージを受信し、応答を送信すること。
  メッセージはプログラムにあらかじめ作成された内容、または直接入力したものとする
- サーバとクライアントが同一 PC 上にあってもよい

- 実装リポジトリ: https://github.com/shapelayer/simple-irc

ウェブサーバの C 言語実装

- ✓ 実装内容

サーバ / main.c

`main.c` / プログラムのエントリポイント: メインループの呼び出し箇所

```
int main(int argc, char const **argv)
{
  // コマンドライン引数のパース
  main_args_t args = parse_args(argc, argv);

  // サーバセッションオブジェクトの生成
  server_sess_t *server_sess;
  server_sess = calloc(1, sizeof(server_sess_t));
  if (!server_sess) PANIC("サーバセッションの割り当てに失敗しました");
  init_server_sess(server_sess);
  printf("Server session initialized.\n");
```

サーバ / main.c

main.c

```c
// サーバポート設定 (setter)
server_sess_set_port(server_sess, args.port);
printf("Starting server on port %d...\n", args.port);
server_sess_open(server_sess);

// メインループ開始
while (1) server_sess_update(server_sess);

return 0;
```

サーバ / server_sess.h

`server_sess.h` / サーバセッションオブジェクトの宣言

```
struct server_sess
{
  int fd;
  int opt;
  int sd_cnt;
  int port;
  struct sockaddr_in address;
  int addrlen;
  client_desc_t clients[];
  fd_set readfds;

  ...
} typedef server_sess_t;
```

`server_sess_t` 構造体の内部フィールドについて

- ✓ `__` で始まるフィールドは、関数内部で一時的に使用される値
- ✓ `server_sess_t` オブジェクト自体とは密接な関連はない
- ✓ `update` 関数内で毎回生成することも可能だが、変数の生成コストが発生する
- ✓ 初回のみ生成し、変数を再利用する構成をとっている

サーバ / server_sess.h

server_sess.h

```
struct server_sess
{
  ...
  // 内部フィールド
  int __new_socket;
  int __valread;
  int __activity;
  char __str_buf[STRING_BUFFER_SIZE];
} typedef server_sess_t;
```

サーバセッションオブジェクトのメソッド宣言

```
// コンストラクタ
void init_server_sess(server_sess_t *sess);

// メインループ
void server_sess_update(server_sess_t *sess);

// サーバのオープン
void server_sess_open(server_sess_t *sess);

// ポート設定
void server_sess_set_port(server_sess_t *sess, int port);

// メッセージのブロードキャスト
void server_sess_broadcast(server_sess_t *sess, int sender_idx, const char *msg);

// クライアントメッセージの処理
void server_sess_handle_client_message(server_sess_t *sess, int index, char *buffer);
```

サーバ / server_sess.c

```
// コンストラクタ
void init_server_sess(server_sess_t *sess)
{
  for (int i = 0; i < SERVER_MAX_CLIENT_CONNECTIONS; i++)
  {
    sess->clients[i].socket = 0;
    snprintf(sess->clients[i].nickname, sizeof(sess->clients[i].nickname), "Guest%d",
i);
  }

  // ソケットのオープンおよびファイルディスクリプタの割り当て試行
  if ((sess->fd = socket(AF_INET, SOCK_STREAM, 0)) == 0)
  {
    PANIC("Error during opening the socket (socket)");
  }
...
```

サーバ / server_sess.c

```
...
  // ソケットオプションの設定試行
  sess->opt = 1;
  if (setsockopt(sess->fd, SOL_SOCKET, SO_REUSEADDR, &sess->opt, sizeof(sess->opt)))
  {
    PANIC("Error during setting the socket's option (setsockopt)");
  }

  // サーバアドレス、ポートデータの設定試行
  sess->address.sin_family = AF_INET;
  sess->address.sin_addr.s_addr = INADDR_ANY;
  server_sess_set_port(sess, SERVER_PORT);
}
```

サーバ / server_sess.c

```c
// サーバのオープン
void server_sess_open(server_sess_t *sess)
{
  // server_sess_tのアドレス情報を用いてサーバのオープンを試行
  if (bind(sess->fd, (struct sockaddr *)&sess->address, sizeof(sess->address)) < 0)
  {
    PANIC("Error during binding the socket (bind)");
  }
  sess->addrlen = sizeof(sess->address);

  if (listen(sess->fd, 3) < 0)
  {
    PANIC("Error during starting listening on the socket (listen)");
  }

  printf("Simple Chat Server started on port %d\n", sess->port);
}
```

サーバ / server_sess.c

```c
// サーバのメインループサイクル
void server_sess_update(server_sess_t *sess)
{
  // ファイルディスクリプタ(FD)セットの初期化および監視対象の設定
  FD_ZERO(&sess->readfds);            // 読み取り監視用のFDセットを0で初期化
  FD_SET(sess->fd, &sess->readfds);   // サーバのリッスンソケットを監視対象に追加
  sess->sd_cnt = sess->fd;            // 最大のソケット番号をサーバソケットに初期設定

  // 現在接続されているすべてのクライアントソケットを監視対象に追加
  for (int i = 0; i < SERVER_MAX_CLIENT_CONNECTIONS; i++)
  {
    int sd = sess->clients[i].socket;
    if (sd > 0) FD_SET(sd, &sess->readfds);
    if (sd > sess->sd_cnt) sess->sd_cnt = sd;
  }
  // アクティビティのあるソケットを監視 (ブロッキング状態)
  // 第3, 4, 5引数がNULLのため、読み取りイベントのみを無期限に待機
  sess->__activity = select(sess->sd_cnt + 1, &sess->readfds, NULL, NULL, NULL);
  if (sess->__activity < 0) PANIC("Error during select operation (select)");
...
```

サーバ / server_sess.c

```
...
  // 新規クライアントの接続要求の処理
  // サーバリッスンソケットに読み取りイベントが発生すれば、新規接続があることを意味する
  if (FD_ISSET(sess->fd, &sess->readfds))
  {
    if ((sess->__new_socket = accept(sess->fd, (struct sockaddr *)&sess->address,
(socklen_t *)&sess->addrlen)) < 0) PANIC();
    // 空いているクライアントスロットを探し、新しいソケット情報を保存
    for (int i = 0; i < SERVER_MAX_CLIENT_CONNECTIONS; i++)
    { if (sess->clients[i].socket == 0) {
        sess->clients[i].socket = sess->__new_socket;
        printf("New connection: Guest%d\n", i);

        char welcome[STRING_BUFFER_SIZE];
        snprintf(welcome, sizeof(welcome), "Welcome! You are Guest%d. Use '/nick
<name>' to change nickname.\n", i);
        send_to_socket(sess->__new_socket, welcome);
        break;
  }}}
...
```

サーバ / server_sess.c

```
...
  // 既存クライアントからのメッセージ受信処理
  for (int i = 0; i < SERVER_MAX_CLIENT_CONNECTIONS; i++)
  {
    int sd = sess->clients[i].socket;
    // 該当クライアントソケットに読み取りイベントが発生したか確認
    if (FD_ISSET(sd, &sess->readfds))
    {
      memset(sess->__str_buf, 0, STRING_BUFFER_SIZE);
      // データの読み取り試行
      if ((sess->__valread = read(sd, sess->__str_buf, STRING_BUFFER_SIZE)) == 0) {
        close(sd); sess->clients[i].socket = 0;
        printf("User disconnected: %s\n", sess->clients[i].nickname);
      } else {
        if (sess->__valread >= STRING_BUFFER_SIZE)
          sess->__str_buf[STRING_BUFFER_SIZE - 1] = '\0';
        else
          sess->__str_buf[sess->__valread] = '\0';
        server_sess_handle_client_message(sess, i, sess->__str_buf);
}}}}
```

サーバ / server_sess.c

```c
// オープンするポートの設定
void server_sess_set_port(server_sess_t *sess, int port)
{
  sess->address.sin_port = htons(port);
  sess->port = port;
}
```

```c
// サーバ全体にメッセージを送信
void server_sess_broadcast(server_sess_t *sess, int sender_idx, const char *msg)
{
  for (int i = 0; i < SERVER_MAX_CLIENT_CONNECTIONS; i++)
    if (sess->clients[i].socket != 0 && i != sender_idx)
      send_to_socket(sess->clients[i].socket, msg);
}
```

サーバ / server_sess.c

```c
// クライアントからメッセージを受信した際
void server_sess_handle_client_message(server_sess_t *sess, int index, char *buffer)
{
  buffer[strcspn(buffer, "\r\n")] = 0;

  if (strlen(buffer) == 0) return;

  // デバッグ目的でコマンド機能(ユーザ名の変更)を追加
  if (COMMAND_NICK_IS_TRIGGERED(buffer))
  {
    command_serv_handle_nick(sess, index, buffer); return;
  }

  char msg_out[STRING_BUFFER_SIZE + 50];
  snprintf(msg_out, sizeof(msg_out), "[%s]: %s\n", sess->clients[index].nickname,
buffer);

  SERVER_SESS_LOG_AND_BRODCAST(sess, index, msg_out);
}
```

サーバ / ビルド

- ✓ 実装は CMake を利用してビルド

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(simple_irc_server C)
set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(HEADERS
  src/command.h     src/server_sess.h   src/socket.h
  include/client.h  include/defines.h   include/panic.h
)
set(SOURCES
  src/command.c src/server_sess.c src/socket.c src/main.c
)
add_executable(${PROJECT_NAME} ${SOURCES} ${HEADERS})
target_include_directories(${PROJECT_NAME} PRIVATE include)
```

サーバ・クライアント共通要素

client.h

```
struct client_desc {
  int socket;
  char nickname[32];
} typedef client_desc_t;
```

サーバ・クライアント共通要素

panic.h

```
#ifndef _SIMPLE_IRC_PANIC_H_
#define _SIMPLE_IRC_PANIC_H_

#define PANIC(msg) { \
  perror(msg); \
  exit(EXIT_FAILURE); \
}

#endif
```

クライアント / main.c

```c
int main(int argc, char const **argv)
{
  // クライアントセッションオブジェクトの生成
  client_sess_t *client_sess;
  client_sess = calloc(1, sizeof(client_sess_t));
  if (!client_sess) PANIC("Failed to allocate client session");
  init_client_sess(client_sess);

  // サーバ接続の試行
  client_sess_connect(client_sess);

  // サーバ接続中のループ
  while (1) client_sess_update(client_sess);

  return 0;
}
```

クライアント / client_sess.h

client_sess.h

```
struct client_sess
{
  int sock;                 // ソケットディスクリプタ
  struct sockaddr_in addr; // ソケットアドレス構造体
  char nickname[32];        // ニックネーム機能に対応
  fd_set readfds;

  int __valread;
  char __str_buf[];
} typedef client_sess_t;
```

クライアント / client_sess.h

```c
// コンストラクタ
void init_client_sess(client_sess_t *sess);

// 現在のユーザ名の変更
void client_sess_set_nickname(client_sess_t *sess, const char *nickname);

// setter: 接続先アドレスの変更
void client_sess_set_addr(client_sess_t *sess, const char *addr, int port);

// サーバへの接続
void client_sess_connect(client_sess_t *sess);

// メインループ
void client_sess_update(client_sess_t *sess);

// サーバ切断時の動作定義
void client_sess_on_disconnected(client_sess_t *sess);

// サーバからの切断
void client_sess_disconnect(client_sess_t *sess);
```

クライアント / client_sess.c

```c
// クライアントセッションコンストラクタ
void init_client_sess(client_sess_t *sess)
{
  sess->sock = -1;
  client_sess_set_nickname(sess, "Guest");

  if ((sess->sock = socket(AF_INET, SOCK_STREAM, 0)) < 0)
    PANIC("Error during socket creation (socket)");

  sess->addr.sin_family = AF_INET;
  client_sess_set_addr(sess, "127.0.0.1", SERVER_PORT);
}
```

クライアント / client_sess.c

```c
// setter: ユーザ名の設定
void client_sess_set_nickname(client_sess_t *sess, const char *nickname)
{
  memset(sess->nickname, 0, sizeof(sess->nickname));
  strncpy(sess->nickname, nickname, sizeof(sess->nickname) - 1);
}
```

クライアント / client_sess.c

```c
// setter: 接続先アドレスの設定
void client_sess_set_addr(client_sess_t *sess, const char *addr, int port)
{
  sess->addr.sin_port = htons(port);

  if (inet_pton(AF_INET, addr, &sess->addr.sin_addr) <= 0)
    PANIC("Invalid address. Address not supported (inet_pton)");
}
```

クライアント / client_sess.c

```c
// サーバへの接続
void client_sess_connect(client_sess_t *sess)
{
  if (connect(sess->sock, (struct sockaddr *)&sess->addr, sizeof(sess->addr)) < 0)
    PANIC("Connection Failed (connect)");

  printf("Connected to server!\n");
  printf("Commands: /nick <name> to change nickname.\n");
}
```

クライアント / client_sess.c

```c
// クライアントメインループ
void client_sess_update(client_sess_t *sess)
{
  // ファイルディスクリプタセットの初期化および監視対象の設定
  FD_ZERO(&sess->readfds);
  FD_SET(STDIN_FILENO, &sess->readfds);
  FD_SET(sess->sock, &sess->readfds);

  // ソケットアクティビティの監視（ブロッキング状態）
  if (select(sess->sock + 1, &sess->readfds, NULL, NULL, NULL) < 0)
  {
    PANIC("Error during select operation (select)");
  }
  ...
```

クライアント / client_sess.c

```
// メッセージ受信時
if (FD_ISSET(sess->sock, &sess->readfds))
{
  memset(sess->__str_buf, 0, STRING_BUFFER_SIZE);
  sess->__valread = read(sess->sock, sess->__str_buf, STRING_BUFFER_SIZE);

  // サーバからの切断時の処理
  if (sess->__valread == 0)
  {
    client_sess_on_disconnected(sess);
    return;
  }

  // 受信したメッセージの出力
  printf("%s", sess->__str_buf);
}
...
```

クライアント / client_sess.c

```
  ...
  // メッセージ入力処理
  if (FD_ISSET(0, &sess->readfds))
  {
    memset(sess->__str_buf, 0, STRING_BUFFER_SIZE);
    // 標準入力からメッセージを読み取り
    if (fgets(sess->__str_buf, STRING_BUFFER_SIZE, stdin) != NULL)
    {
      send(sess->sock, sess->__str_buf, strlen(sess->__str_buf), 0);
    }
  }
}
```

クライアント / client_sess.c

```c
// サーバ切断時の動作定義
void client_sess_on_disconnected(client_sess_t *sess)
{
  printf("Disconnected from server.\n");
  exit(EXIT_SUCCESS);
}
```

```c
// サーバからの切断
void client_sess_disconnect(client_sess_t *sess)
{
  // ファイルディスクリプタを閉じる
  close(sess->sock);
}
```

実行結果

```
./simple_irc_client
❯ ./simple_irc_client
Connected to server!
Commands: /nick <name> to change nickname.
Welcome! You are Guest1. Use '/nick <name>' to change nickname.
ぷにぷに
[Guest0]: ぷにぷにって言うな！
[Guest0]: おい！赤スーパーだったからって、ぷにぷにって言えば許されるとでも思ってるのか！他に言うこ
とがあるだろ！
[Guest0]: おいなんかいうことがあったろ
…でもかわいい
[Guest0]: あ、いいねいいね
[Guest0]: 気分がよくなってきた
かわいいでもぷにぷに…
[Guest0]: …何か意味違うそう
```

実行結果

./simple_irc_server

```
❯ ./simple_irc_server
Server session initialized.
Starting server on port 6667...
Simple Chat Server started on port 6667
New connection: Guest0
New connection: Guest1
[Server] Guest0 changed name to Duda
[Server] Guest1 changed name to John Kirby
[Duda]: Korea is helpful in Ukraine's war against Russia bec
ause of its ammunition deliveries, as its huge supply of art
illery shells.
[Duda]: I didn't think this was "going to be possible withou
t the intervention of the United States," as Korea is "fearf
ul" of Russian and Chinese reactions.
[Duda]: US have to intervene with Korea on Ukrainian arms is
sue
[John Kirby]: We have a treaty alliance with South Korea.
[John Kirby]: We take that commitment very, very seriously.
The U.S. is grateful for the support South Korea has already
 provided to Ukraine.
```

```
❯ ./simple_irc_client
Connected to server!
Commands: /nick <name> to change nickname.
Welcome! You are Guest0. Use '/nick <name>' to change nickna
me.
/nick Duda
[Server] Nickname changed to Duda
[Server] Guest1 changed name to John Kirby
Korea is helpful in Ukraine's war against Russia because of
its ammunition deliveries, as its huge supply of artillery s
hells.
I didn't think this was "going to be possible without the in
tervention of the United States," as Korea is "fearful" of R
ussian and Chinese reactions.
US have to intervene with Korea on Ukrainian arms issue
[John Kirby]: We have a treaty alliance with South Korea.
[John Kirby]: We take that commitment very, very seriously.
The U.S. is grateful for the support South Korea has already
 provided to Ukraine.
```

```
❯ ./simple_irc_client
Connected to server!
Commands: /nick <name> to change nickname.
Welcome! You are Guest1. Use '/nick <name>' to change nickna
me.
[Server] Guest0 changed name to Duda
/nick John Kirby
[Server] Nickname changed to John Kirby
[Duda]: Korea is helpful in Ukraine's war against Russia bec
ause of its ammunition deliveries, as its huge supply of art
illery shells.
[Duda]: I didn't think this was "going to be possible withou
t the intervention of the United States," as Korea is "fearf
ul" of Russian and Chinese reactions.
[Duda]: US have to intervene with Korea on Ukrainian arms is
sue
We have a treaty alliance with South Korea.
We take that commitment very, very seriously. The U.S. is gr
ateful for the support South Korea has already provided to U
kraine.
^[
```

実行結果

./simple_irc_server

```
❯ ./simple_irc_client
zsh: no such file or directory: ./simple_irc_client
❯ ./simple_irc_server
Server session initialized.
Starting server on port 6667...
Simple Chat Server started on port 6667
New connection: Guest0
New connection: Guest1
[Guest0]: 저 뭐 하나만 여쭤봐도 되겠습니까?
[Guest1]: 네네네
[Guest0]: 저희가 이걸 좀 촉박하게 제안을 드렸잖아요?
[Guest1]: 네네
[Guest0]: 그런데 오늘 보니까 PPT가 70장이 넘었는데
[Guest0]: 자..혹시.. 평소에도 이런 생각하고 계신가요?
[Guest0]: 급하게 만들었다기에는 너무나도 일관성있고 해서
[Guest0]: 혹시 이게 압둘씨께서 평상시에 이런걸 막 미리 준비
하신게 아닐까
[Guest1]: 일관성이 있다라는 것은 무엇인가?
[Guest1]: 이 모든 내용이 참이라는 것을 증명하는 것 아니겠습
니까?
[Guest0]: 아..;;
[Guest0]: 방송인이 되어주셔서 정말 감사합니다..;;
[Guest0]: 사이비 종교같은거 안 만들어주셔서 정말 감사합니다;
;
[Guest1]: 예예 항상 감사하십시오
```

```
❯ ./simple_irc_client
Connected to server!
Commands: /nick <name> to change nickname.
Welcome! You are Guest0. Use '/nick <name>' to change nickna
me.
저 뭐 하나만 여쭤봐도 되겠습니까?
[Guest1]: 네네네
저희가 이걸 좀 촉박하게 제안을 드렸잖아요?
[Guest1]: 네네
그런데 오늘 보니까 PPT가 70장이 넘었는데
자..혹시.. 평소에도 이런 생각하고 계신가요?
급하게 만들었다기에는 너무나도 일관성있고 해서
혹시 이게 압둘씨께서 평상시에 이런걸 막 미리 준비하신게 아닐
까
[Guest1]: 일관성이 있다라는 것은 무엇인가?
[Guest1]: 이 모든 내용이 참이라는 것을 증명하는 것 아니겠습
니까?
아..;;
방송인이 되어주셔서 정말 감사합니다..;;
사이비 종교같은거 안 만들어주셔서 정말 감사합니다;;
[Guest1]: 예예 항상 감사하십시오
```

```
❯ ./simple_irc_client
Connected to server!
Commands: /nick <name> to change nickname.
Welcome! You are Guest1. Use '/nick <name>' to change nickna
me.
[Guest0]: 저 뭐 하나만 여쭤봐도 되겠습니까?
네네네
[Guest0]: 저희가 이걸 좀 촉박하게 제안을 드렸잖아요?
네네
[Guest0]: 그런데 오늘 보니까 PPT가 70장이 넘었는데
[Guest0]: 자..혹시.. 평소에도 이런 생각하고 계신가요?
[Guest0]: 급하게 만들었다기에는 너무나도 일관성있고 해서
[Guest0]: 혹시 이게 압둘씨께서 평상시에 이런걸 막 미리 준비
하신게 아닐까
일관성이 있다라는 것은 무엇인가?
이 모든 내용이 참이라는 것을 증명하는 것 아니겠습니까?
[Guest0]: 아..;;
[Guest0]: 방송인이 되어주셔서 정말 감사합니다..;;
[Guest0]: 사이비 종교같은거 안 만들어주셔서 정말 감사합니다;
;
예예 항상 감사하십시오
```